PBLE Report

Semester	S.E. Semester III – Computer Engineering
Subject	Analysis of Algorithm
Subject Professor In-charge	Dr. Swapnil S. Sonawane

Roll Number	Name of Students
26102B2009	Sanskruti Sachin Shinde

Name of the Project:

Smart Delivery Planning Using Fractional Knapsack Algorithm

Project Description:

The Fractional Knapsack Problem is a classic optimization problem that is solved using the Greedy Method. In this project, a delivery vehicle has a fixed carrying capacity and a set of packages, each having its own value (profit) and weight. The aim is to load the vehicle so that the total value of the carried packages is as large as possible.

The program is written in C and is menu-driven. It accepts the package details, calculates the Value/Weight ratio of every package, and sorts the packages in decreasing order of this ratio. Packages with the highest ratio are loaded first. If a package does not fit completely in the remaining capacity, only the required fraction of it is taken. This approach guarantees the maximum possible value for the given capacity.

Objectives

1. To understand and implement the Greedy Algorithm.
2. To calculate the Value/Weight ratio of each package.
3. To sort the packages in decreasing order of ratio.
4. To select complete and fractional packages optimally.
5. To determine the maximum obtainable value for the vehicle.
6. To analyze the time complexity of the algorithm.

Problem Statement

A delivery vehicle has a maximum carrying capacity of W units. There are n packages, where the i-th package has a value v_i and a weight w_i. Packages may be divided, so any fraction x_i ($0 \leq x_i \leq 1$) of a package can be loaded. The problem is to find the fractions x_i so that the total value is maximized:

$$\textbf{Maximize } \Sigma (x_i \times v_i) \quad \textbf{subject to} \quad \Sigma (x_i \times w_i) \leq W$$

Algorithm

1. Read the number of packages n.
2. Input the value and weight of each package.
3. Input the vehicle capacity.
4. Calculate the Value/Weight ratio for every package.
5. Sort the packages in decreasing order of ratio.
6. Initialize the remaining capacity as the vehicle capacity, and the total value as 0.
7. Take the packages one by one in sorted order.
8. If the weight of the package is less than or equal to the remaining capacity, select the whole package, add its value to the total, and reduce the remaining capacity.
9. Otherwise, select only the fraction (remaining capacity / weight) of the package, add the proportional value, and stop.
10. Display the selected packages, the total weight used and the maximum value obtained.

Input

Package	Value	Weight
1	40	2
2	90	4
3	100	6
4	20	1

Vehicle Capacity = 15

Ratio Calculation

Ratio = Value / Weight

Package	Value	Weight	Ratio
1	40	2	20.00
2	90	4	22.50
3	100	6	16.67
4	20	1	20.00

Sorted Order by Ratio

Order	Package	Ratio
1	Package 2	22.50
2	Package 1	20.00
3	Package 4	20.00
4	Package 3	16.67

Output / Result

Selected Packages

Package	Fraction Selected
1	1.00
2	1.00
3	1.00
4	1.00

Total Weight Used = 13.00

Maximum Value Obtained = 250.00

The total weight of all four packages is 2 + 4 + 6 + 1 = 13, which is less than the vehicle capacity of 15. Hence all packages are loaded completely, and 2 units of capacity remain unused. The maximum value is 40 + 90 + 100 + 20 = 250.

Time Complexity

Overall Time Complexity: O(n log n)

Justification:

Step	Complexity
Ratio calculation (one pass over n packages)	O(n)
Sorting packages by ratio	O(n log n)
Greedy selection (one pass over sorted list)	O(n)

Since sorting is the most expensive step, it dominates the running time. Therefore, the total time complexity is O(n) + O(n log n) + O(n) = **O(n log n)**.

Advantages

1. The greedy strategy gives an optimal solution for the fractional version of the problem.
2. The algorithm is simple to understand and easy to implement.
3. It is efficient, with a running time of O(n log n).
4. The vehicle capacity is used to the fullest when packages are divisible.
5. It works for any number of packages and any capacity.

Applications

1. Delivery and logistics planning (loading vehicles for maximum profit).
2. Resource allocation and budget distribution.
3. Cargo loading in ships, trucks and aircraft.
4. Bandwidth and CPU time allocation in computer systems.
5. Investment planning, where money can be split among different options.

Conclusion

The Fractional Knapsack Algorithm successfully maximizes the total value of packages within the carrying capacity of the vehicle. The Greedy approach selects packages according to the highest Value/Weight ratio and gives an optimal solution. For the given input, all four packages fit within the capacity of 15, giving a total weight of 13 and a maximum value of 250. The project shows the practical use of Greedy Algorithms in resource allocation and delivery planning, and the time complexity of O(n log n) was verified through analysis of the program.

Project Code:

```
#include <stdio.h>

#define MAX 100

int n;
float value[MAX], weight[MAX], ratio[MAX], selected[MAX];
float capacity, maxValue = 0, totalWeight = 0;

void enterDetails()
{
    int i;
```

```
    printf("Enter number of packages: ");
    scanf("%d", &n);

    if (n <= 0 || n > MAX)
    {
        printf("Invalid number of packages. Please enter a value between 1 and %d.\n", MAX);
        n = 0;
        return;
    }

    for (i = 0; i < n; i++)
    {
        printf("\nPackage %d\n", i + 1);
        printf("Enter value: ");
        scanf("%f", &value[i]);

        printf("Enter weight: ");
        scanf("%f", &weight[i]);

        if (weight[i] <= 0)
        {
            printf("Weight must be positive. Please re-enter package %d.\n", i + 1);
            i--;
        }
        else
        {
            selected[i] = 0;
            ratio[i] = value[i] / weight[i];
        }
    }

    printf("\nEnter vehicle capacity: ");
    scanf("%f", &capacity);

    if (capacity <= 0)
    {
        printf("Capacity must be positive.\n");
        capacity = 0;
    }
}

void displayDetails()
{
    int i;

    if (n <= 0)
    {
        printf("\nNo package details available. Please enter details first.\n");
        return;
    }

    printf("\n-------------------------------------------------\n");
    printf("Package\tValue\tWeight\tRatio\n");
    printf("-------------------------------------------------\n");

    for (i = 0; i < n; i++)
    {
        if (weight[i] > 0)
            ratio[i] = value[i] / weight[i];
        else
            ratio[i] = 0;

        printf("%d\t%.2f\t%.2f\t%.2f\n", i + 1, value[i], weight[i], ratio[i]);
    }
}

void calculateRatio()
{
    int i;

    if (n <= 0)
    {
        printf("\nNo package details available.\n");
        return;
    }
```

```
    for (i = 0; i < n; i++)
    {
        if (weight[i] > 0)
            ratio[i] = value[i] / weight[i];
        else
            ratio[i] = 0;
    }

    printf("\nValue/Weight Ratios Calculated Successfully.\n");
}

void sortPackages()
{
    int i, j;
    float temp;

    if (n <= 0)
    {
        printf("\nNo package details available.\n");
        return;
    }

    for (i = 0; i < n - 1; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            if (ratio[i] < ratio[j])
            {
                temp = ratio[i];
                ratio[i] = ratio[j];
                ratio[j] = temp;

                temp = value[i];
                value[i] = value[j];
                value[j] = temp;

                temp = weight[i];
                weight[i] = weight[j];
                weight[j] = temp;
            }
        }
    }

    printf("\nPackages Sorted in Decreasing Order of Ratio.\n");
}

void findMaximumValue()
{
    int i;
    float remaining = capacity;

    if (n <= 0)
    {
        printf("\nNo package details available.\n");
        return;
    }

    maxValue = 0;
    totalWeight = 0;

    for (i = 0; i < n; i++)
        selected[i] = 0;

    for (i = 0; i < n; i++)
    {
        if (weight[i] <= 0)
            continue;

        if (weight[i] <= remaining)
        {
            selected[i] = 1.0;
            remaining -= weight[i];
            totalWeight += weight[i];
            maxValue += value[i];
        }
        else
        {
            selected[i] = remaining / weight[i];
```

```
            totalWeight += remaining;
            maxValue += value[i] * selected[i];
            remaining = 0;
            break;
        }
    }

    printf("\nMaximum Value = %.2f\n", maxValue);
    printf("Time Complexity: O(n log n) because sorting dominates the work, and the greedy selection
runs in O(n).\n");
}

void displaySelectedPackages()
{
    int i;

    if (n <= 0)
    {
        printf("\nNo package details available.\n");
        return;
    }

    printf("\n-----------------------------------------\n");
    printf("Package\tSelected Fraction\n");
    printf("-----------------------------------------\n");

    for (i = 0; i < n; i++)
    {
        if (selected[i] > 0)
            printf("%d\t%.2f\n", i + 1, selected[i]);
    }

    printf("\nTotal Weight Used = %.2f\n", totalWeight);
    printf("Maximum Value Obtained = %.2f\n", maxValue);
}

int main()
{
    int choice;

    do
    {
        printf("\n\n===== FRACTIONAL KNAPSACK MENU =====\n");
        printf("1. Enter Package Details\n");
        printf("2. Display Package Details\n");
        printf("3. Calculate Value/Weight Ratio\n");
        printf("4. Sort Packages by Ratio\n");
        printf("5. Find Maximum Value\n");
        printf("6. Display Selected Packages\n");
        printf("7. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                enterDetails();
                break;

            case 2:
                displayDetails();
                break;

            case 3:
                calculateRatio();
                break;

            case 4:
                sortPackages();
                break;

            case 5:
                findMaximumValue();
                break;

            case 6:
```

```
                displaySelectedPackages();
                break;

            case 7:
                printf("Exiting Program...\n");
                break;

            default:
                printf("Invalid Choice!\n");
        }

    } while (choice != 7);

    return 0;
}
```

Output Screenshots:

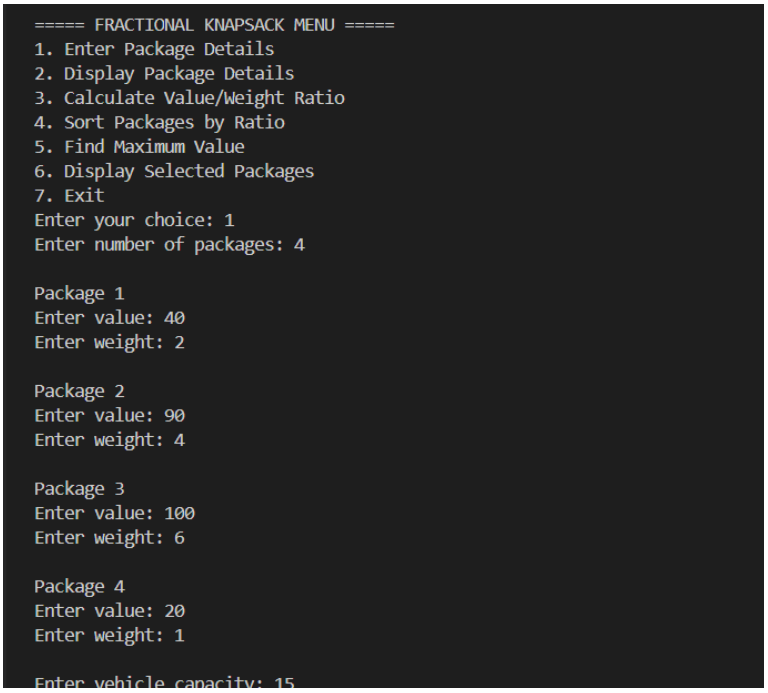


Fig. 1: Package Details Entry

```
===== FRACTIONAL KNAPSACK MENU =====
1. Enter Package Details
2. Display Package Details
3. Calculate Value/Weight Ratio
4. Sort Packages by Ratio
5. Find Maximum Value
6. Display Selected Packages
7. Exit
Enter your choice: 2

------------------------------------------------
Package Value   Weight  Ratio
------------------------------------------------
1       40.00   2.00    20.00
2       90.00   4.00    22.50
3       100.00  6.00    16.67
4       20.00   1.00    20.00
```

Fig. 2: Display Package Details

```
===== FRACTIONAL KNAPSACK MENU =====
1. Enter Package Details
2. Display Package Details
3. Calculate Value/Weight Ratio
4. Sort Packages by Ratio
5. Find Maximum Value
6. Display Selected Packages
7. Exit
Enter your choice: 3

Value/Weight Ratios Calculated Successfully.
```

Fig. 3: Value/Weight Ratio Calculation

```
===== FRACTIONAL KNAPSACK MENU =====
1. Enter Package Details
2. Display Package Details
3. Calculate Value/Weight Ratio
4. Sort Packages by Ratio
5. Find Maximum Value
6. Display Selected Packages
7. Exit
Enter your choice: 4

Packages Sorted in Decreasing Order of Ratio.
```

Fig. 4: Sorted Packages by Ratio

```
===== FRACTIONAL KNAPSACK MENU =====
1. Enter Package Details
2. Display Package Details
3. Calculate Value/Weight Ratio
4. Sort Packages by Ratio
5. Find Maximum Value
6. Display Selected Packages
7. Exit
Enter your choice: 5

Maximum Value = 250.00
Time Complexity: O(n log n) because sorting dominates the work, and the greedy selection runs in O(n).
```

Fig. 5: Maximum Value Calculation

```
===== FRACTIONAL KNAPSACK MENU =====
1. Enter Package Details
2. Display Package Details
3. Calculate Value/Weight Ratio
4. Sort Packages by Ratio
5. Find Maximum Value
6. Display Selected Packages
7. Exit
Enter your choice: 6

-----------------------------------------
Package Selected Fraction
-----------------------------------------
1       1.00
2       1.00
3       1.00
4       1.00

Total Weight Used = 13.00
Maximum Value Obtained = 250.00
```

Fig. 6: Selected Packages and Final Output

```
===== FRACTIONAL KNAPSACK MENU =====
1. Enter Package Details
2. Display Package Details
3. Calculate Value/Weight Ratio
4. Sort Packages by Ratio
5. Find Maximum Value
6. Display Selected Packages
7. Exit
Enter your choice: 7
Exiting Program...
PS C:\Users\Sanskruti Shinde\AOA PBLE>
```

Fig. 7: Exit Screen

GitHub Repository Link of Project:

https://github.com/sanskruti631/Fractional-Knapsack-PBLE